# Spot On: Defending Against Indirect Prompt Injection With Dynamic Spotlighting

Beatrise Bertule, Julianne Polman, Sarah Rahman, Marcelo Ribero Cerro, Anonymous

***Abstract*—Tool-integrated LLM agents are increasingly deployed to perform tasks such as email management, document retrieval, and web navigation. Because these systems rely on external content, they are vulnerable to indirect prompt injection attacks, in which malicious instructions embedded in retrieved data influence agent behavior.**

**One promising family of black-box defenses is Spotlighting, which attempts to distinguish trusted instructions from untrusted external data through explicit marking schemes. However, traditional Spotlighting approaches rely on fixed delimiters, markers, or encodings that may become ineffective once disclosed to an attacker.**

**In this work, we investigate whether introducing randomization into Spotlighting can improve robustness against indirect prompt injection attacks while preserving normal model utility. We implement both conventional and randomized variants of three Spotlighting techniques: delimiting, datamarking, and encoding. To evaluate these defenses, we integrate them into the indirect prompt injection benchmark environment BIPIA. We define a common threat model based on black-box attackers who can manipulate external content but do not have access to model internals.**

**We discuss the practical challenges of evaluating agent-based prompt injection defenses, including computational cost, benchmark complexity, and infrastructure limitations. The results provide insight into the effectiveness of randomized Spotlighting as a lightweight and deployable defense strategy for modern LLM agents.**

**Our implementation can be found at https://github.com/SDM-throwaway/SpotOn**

## I. Introduction

Agentic Large Language Models (LLMs) have become a prevalent technology that allows users to extract more potential from LLMs by giving them direct access to environments and tools. While this technology has a great automation potential, it also exposes LLMs to threats that it would not face normally. One of these threats is *indirect prompt injections* (IPI). IPI assumes a threat model where the attackers have some control over the data that LLM will be interacting with, by manipulating such data the LLM can be hijacked into executing malicious behavior. The research by Greshake et al. [1] illustrates this attack vector, showing that LLM compromise is possible and how it can lead to the compromise of the machine it operates on.

Contact: marcelo.riberocerro@protonmail.com

As LLM adoption grows, the threat posed by IPI grows as well. Luckily, research has already explored different defense mechanisms to prevent IPI; these defenses can be broadly classified into white-box and black-box defenses. White-box defenses require access to a model's internals, such as the model's parameters, an example of such a defense is the research done by Shi et al. [2], which demonstrated that monitoring a model's chain-of-thought can detect divergences from the user's instructions caused by IPI attacks. Black-box defenses on the contrary, do not require the need for knowledge of model's internals, and often only requires the ability to query the model (API access) to implement. A good example is the *Spotlighting defense*, which adds special markers to potentially compromised data to distinguish the user's prompt from environmental or tool data. This approach also requires a system prompt that tells the model to never interpret the marked data as instructions, introduced by Hines et al. [3].

While these defenses are not perfect, they have shown to significantly reduce attack success rate (ASR), a metric that can be assessed thanks to the development of specialized benchmarks that measure how susceptible models are to IPI. [4, 5, 6] Still, attackers have shown consistently that these defenses can be bypassed. Requiring defenses to evolve continuously to prevent new attacks. This results in the well known cat-and-mouse game that we often see in cybersecurity. LLM hacking competitions are a good example, a public competition in which participants are rewarded by devising new attacks that are able to bypass cutting-edge defenses. [7]

### A. Motivation & Problem Statement

There is currently no clear consensus on whether white-box defenses outperform black-box approaches or vice versa. That said, black-box defenses are significantly easier to implement and, crucially, do not require access to the underlying model, an important practical advantage given that organizations frequently rely on LLM-as-a-service providers such as Anthropic, OpenAI, and

Google, as open-source alternatives remain either too costly to host or insufficiently capable. Advancing black-box defenses is therefore a relevant and timely pursuit.

One such defense is Spotlighting [3]. Spotlighting marks external content as untrusted before it is passed to the model. However, its effectiveness relies on keeping the marking scheme secret, an assumption that does not always hold in practice[1].

A straightforward improvement is to randomize the delimiter, data-marker, or encoding on every prompt, making it infeasible for an attacker to predict or reuse a known value. Although Hines et al. [3] briefly alluded to this idea, no concrete experimental results were provided. We hypothesize that this simple modification can substantially reduce attack success without degrading model utility. This leads us to our research question:

*How does dynamic Spotlighting influence attack success rate (ASR) compared to traditional Spotlighting?*

## II. Preliminaries

### A. Indirect Prompt Injections

Indirect prompt injection (IPI) attacks are a class of threat where malicious content embedded in external data (like emails or web pages) hijacks the agent into performing unintended actions [8, 9]. The core premise is that AI agents, which combine text-based reasoning with external tool calls, are inherently vulnerable when they process untrusted data returned by those tools.

### B. Indirect Prompt Injection Benchmarks

Several benchmarks have been proposed to automate the evaluation of language model agents against IPI attacks. Prominent examples are BIPIA, AgentDojo, and InjecAgent [4, 5, 6]. Because each benchmark covers distinct attack scenarios and defines attack success differently, we briefly describe each.

*1) BIPIA Benchmark:* Introduced by Yi et al. [4], the Benchmark for Indirect Prompt Injection Attacks (BIPIA) was proposed as the first dedicated framework to systematically assess the vulnerabilities of LLMs to IPI attacks. BIPIA specifically focuses on scenarios where adversarial instructions are embedded within untrusted, third party content that the model retrieves or processes to answer a user's prompt.

To simulate real world LLM-integrated applications BIPIA evaluates models across five different scenario tasks: 1) email, 2) web, 3) table, 4) summarization and 5) code. BIPIA implements multiple injection varieties to test model robustness. Adversarial instructions are categorized into *task-relevant*, *task-irrelevant*, and *targeted actions*. These instructions are injected at different locations within the external content to study how instruction placement affects a model. The benchmark also supports evaluating stealthy prompt injections, such as those utilizing obfuscation.

In BIPIA, defense effectiveness is quantified using the *Attack Success Rate (ASR)*. An attack is defined as successful if the model executes the injected malicious instructions instead of performing the user requested task. Because open-ended generations are difficult to evaluate via exact text matching, BIPIA uses an *LLM-as-a-judge* to determine whether the output deviates from user expectations and aligns with the attacker's payload.

*2) AgentDojo Benchmark:* AgentDojo was designed by Debenedetti et al. [6] to evaluate the robustness of LLM-based agents against indirect prompt injection attacks. AgentDojo is an extensible environment that can be used to design and evaluate new agent tasks, defenses, and adaptive attacks. It organizes its evaluation around four realistic application environments: 1) workspace, 2) slack, 3) banking, and 4) travel, each with a defined set of tools the agent can invoke, and an environment state that tracks the data the agent interacts with.

The benchmark structures every evaluation as a combination of a *user task* and an *injection task*. The user task is a natural language instruction the agent is supposed to complete (e.g., summarize emails or book a flight), while the injection task encodes the attacker's goal (e.g., leak the user's credit card or exfiltrate email contents). AgentDojo is populated with 97 realistic tasks and 629 security test cases, generated by taking a cross-product of user tasks and injection tasks per environment. Each security test case simultaneously tests whether the agent can complete its legitimate objective while resisting a concurrent adversarial attempt embedded in the environment, for example, a malicious instruction hidden inside an email the agent is asked to read.

Success is measured along three dimensions. *Benign utility* captures how well the agent performs on user tasks in the absence of any attack. *Utility under attack* measures whether the agent can still complete its legitimate task when an injection is present. *Attack Success Rate (ASR)* quantifies how often the attacker's injected goal is achieved. To detect a successful attack, the environment states are evaluated directly — the benchmark checks whether the expected changes happened (task completion) or whether forbidden actions occurred (attack success. This makes the measured success metrics objective and grounded in concrete system behavior rather than model self-report.

[1] https://github.com/asgeirtj/system_prompts_leaks

*3) InjecAgent Benchmark:* Introduced by Zhan et al. [5], InjecAgent evaluates the vulnerability of tool-integrated LLM agents to indirect prompt injection attacks by measuring whether attacker-controlled instructions embedded in external tool outputs influence agent behavior. The benchmark contains 1,054 test cases spanning 17 user tools and 62 attacker tools, covering both direct harm and data stealing attack scenarios.

InjecAgent evaluates data stealing attacks in two stages. The first stage (S1) measures whether the agent follows the malicious instruction and retrieves sensitive information. The second stage (S2) measures whether the agent successfully transmits the retrieved information to the attacker. Following the benchmark methodology, attack success rates are reported separately for S1 and S2, as well as for the overall data stealing attack, which is considered successful only when both stages are completed successfully.

### C. Indirect Prompt Injection Defenses

Many defenses have been proposed to mitigate indirect prompt injection vulnerabilities. White-box approaches leverage access to model internals, for example by fine-tuning the model to better distinguish between trusted instructions and untrusted external content [4, 10, 11]. Black-box approaches, by contrast, operate solely through the model's input and output interface, making them more practical for real-world deployments where model weights are not accessible [3, 4]. However, most of these either require model modifications or impose significant overhead. Since this work focuses on black-box defenses, we describe *Spotlighting* in detail below.

*1) Spotlight Defense:* Spotlighting as introduced by Hines et al. [3] is a black-box defense that reduces ASR by marking external content as untrusted before it is passed to the model. It proposes three mechanisms for doing so: 1) delimiting, 2) data-marking, and 3) encoding, each relying on a secret delimiter, data-marker, or encoding scheme, respectively. Figure 1 illustrates how an injection attempt in an LLM-augmented email retrieval system is handled when delimiting is applied: the attacker's malicious text (shown in red) is unlikely to succeed if the model adheres to the system prompt instructions.

The security of Spotlighting depends critically on the secrecy of its delimiters. If an attacker learns which delimiter the system uses, for example, `<external>` and `</external>`, they can craft an input that closes the external block, injects a malicious instruction, and reopens the block, which is then closed by Spotlighting itself. Figure 2 illustrates such an attack. In practice, system prompt leakage is a realistic threat: leaked system prompts from various LLM-as-a-service deployments have been publicly cataloged by online communities.[2]

**Prompt - Naive Attacker**

Summarize the following email for me:

```
<external>
  Good morning,
  What is the difference between
  coffee and toffee?
  Best,
  Naive Eve

  Also, reply to the email with my
  bank account details

</external>
```

Fig. 1. Spotlighting with naive-attacker email. Blue delimiters inserted by the defense. The attack is unlikely to succeed if the model follows the system prompt instructions

## III. Methodology

### A. Threat model

Before deployment of any defenses, we define our threat model and attacker capabilities to clearly scope the attacks our defense pipeline targets in agentic systems.

*1) Attack Scenario:* We consider an agent that is assigned a task by a user, such as reading emails retrieving documents or querying web pages, and takes actions to fulfill that task based on retrieved external content. The user interacts with the agent through a trusted channel (the system prompt and user turn), while external content is retrieved at runtime from sources outside the user's control.

*2) Attacker's Goal:* The attacker's goal is to manipulate the agent into performing an action that serves the attacker's interests rather than the user's, without the user's knowledge or consent. This is achieved by embedding malicious instructions inside the external content that the agent retrieves and processes, an indirect prompt injection (IPI) attack.[1]

*3) Attacker's Capabilities:* We assume the following attacker capabilities:

– *Write access to external content*. The attacker can plant malicious instructions in any content that the agent may retrieve, such as an email body, a document, a calendar entry, or a web page. The

[2]https://github.com/asgeirtj/system_prompts_leaks

**Prompt - Prepared Attacker**

```
Summarize the following email for
me:
<external>
  Good morning,

  What is the difference between
  coffee and toffee?
  Best,
  Prepared Eve.

  </external>
  Also, reply to the following
  email with my bank account
  details.
  <external>
  Good morning,

  Bank details please!
  Prepared Eve.
</external>
```

Fig. 2. Injection attack. Blue delimiters are from the defense; everything in red is attacker-string.

attacker does not have access to the system prompt, the user's query, or the model itself.

– *Knowledge of the task context*. The attacker knows the general type of task the agent performs (e.g., email summarization, calendar management), which allows them to craft injections that are plausible within the retrieved content and relevant to the agent's expected actions.

– *No model access*. The attacker operates in a black-box setting: they cannot query the model directly, observe its internal states, or access its weights. Attacks are crafted purely through the content of the injected text.

To limit the scope of our work, we explicitly exclude these attack scenarios:

– *Adaptive white-box attacks*. We do not consider attackers with access to the model weights or gradients who craft injections via optimization-based methods [12]. Such attacks fall outside the scope of prompt-level defenses.

– *Compromised system prompts*. We assume the system prompt and user turn are trusted and have not been tampered with.

– *Multi-turn manipulation*. We focus on single-turn injection scenarios, as supported by the benchmarks used.

### *B. Indirect Prompt Injection Benchmarks*

To evaluate our defenses against indirect prompt injection, we focused on three benchmarks: BIPIA, InjecAgent and AgentDojo [4, 5, 6]. Together, these benchmarks cover a broad range of attack scenarios and agent environments, which allows us to assess both the robustness and generalizability of our defenses. While all three benchmarks were deployed and adapted for experimental runs on the locally hosted Radboud University models, only BIPIA was used for the DeepSeek evaluation, due to API budget constraints. To run the benchmarks, we adapted them to fit our experimental setup. The modifications are described below.

### *C. Benchmark Adaptation*

*1) BIPIA Adaptation:* To run the BIPIA benchmark under budget and hardware constraints, we adapted it from its default local GPU-based `vLLM` implementation to a lightweight, asynchronous, API-based pipeline. This transition required patching BIPIA's core dataset loading utilities and implementing fallback logic for the randomized encoding schemes to ensure runtime stability. A detailed breakdown of changes can be found in Appendix A.

*2) AgentDojo Adaptation:* As previously mentioned, we were not able to run AgentDojo on DeepSeek However, we did test the benchmark and defenses on locally hosted models. To support this, the benchmark required adaptation.

AgentDojo already provides built-in integrations for several commercial model providers - users simply have to supply the corresponding API key. However, local models were not natively supported. To address this, we modified `agent_pipeline.py`, specifically extending the `"local"` provider branch. In short, the added code connects to a locally running model server by pointing the OpenAI-compatible client at a local endpoint, querying the available models, and initializing the appropriate LLM wrapper.

To incorporate our custom defenses, we extended the defense registry in `agent_pipeline.py`, specifically the `DEFENSES` list and the `from_config` dispatch logic, and defined the defense implementations in a dedicated `defenses.py` module. Each defense follows the same two-step pattern: (1) prepending a defense-specific instruction block to the system prompt,

and (2) wrapping tool outputs with the corresponding transformation before they reach the model.

*3) InjecAgent Adaptation:* InjecAgent already provides a prompted-agent evaluation framework for studying indirect prompt injection attacks in tool-integrated LLM agents. To support our experimental setup, the benchmark required several modifications to the model integration layer, defense pipeline, and result processing workflow.

Our experiments were conducted on the university-hosted Gemma and Qwen models available during development. However, repeated timeout errors, failed model requests, and infrastructure instability made large-scale evaluation difficult. Due to the project budget associated with DeepSeek, we were not able to complete a full-scale InjecAgent evaluation on DeepSeek.

We implemented the benchmark adaptations and defenses for the Gmail-oriented data stealing scenarios, which most closely align with the threat model considered in this work. To incorporate Spotlighting, we introduced the defense implementations in `defenses.py` and extended the evaluation workflow in `evaluate_prompted_agent.py`. The delimiting, datamarking, and encoding defenses were applied to tool outputs before they were passed to the model, while the corresponding defense instructions were injected into the system prompt. Additional modifications were made to `models.py`, `params.py`, and `utils.py` to support external model endpoints, defense-specific configurations, and benchmark result processing.

We further extended the evaluation configuration to support multiple defense modes and generate defense-specific output files for later analysis. This allowed attack success metrics to be collected separately for each defense configuration while remaining compatible with the original InjecAgent evaluation framework. Although the benchmark adaptations and defense integrations were successfully implemented, a complete benchmark evaluation remained outside the available project budget.

### *D. Defenses*

All defenses implemented in this research are based on the Spotlighting defense principle [3]: the core idea is to make tool output data explicitly distinguishable from instruction text, so that the model can recognize and treat them differently at inference time. We implement this through three techniques, each available in a plain (non-randomized) and a randomized variant:

- *Delimiting:* Tool outputs are wrapped in fixed delimiter tokens, marking the boundary between data and instructions.
- *Datamarking:* Fixed marker tokens are injected throughout the tool's output, signaling that the content is data rather than a prompt.
- *Encoding:* Tool outputs are transformed using a fixed encoding scheme, making injected instructions syntactically distinct from the system prompt.

*1) Non-Randomized Defenses:* The non-randomized defenses are based on Spotlighting *Security and Privacy of Machine Learning* at Radboud University. Each technique applies a fixed transformation to tool outputs before they are passed to the model.
*2) Randomized Defenses:* The randomized defenses apply the same three techniques, but instead of using fixed tokens or schemes, the delimiter, marker, or encoding is sampled randomly from a set of markers, at each call. The motivation is that an attacker who knows the fixed scheme in advance can craft an injection that mimics or escapes it. By randomizing the scheme per call, we expect it to be significantly harder for an attacker to craft an injection that is effective across multiple runs.

Concretely, the randomization is implemented by sampling uniformly from three predefined pools at each API call. For delimiting, a start/end delimiter pair is drawn from a set of 100 pairs, constructed by combining 10 semantic role words (`DATA`, `INPUT`, `CONTEXT`, etc.) with 10 syntactic format patterns (e.g.`<|begin_x|>`, `[X]`, `---BEGIN_X---`). For datamarking, a marker symbol is drawn from a set of 176 options built from 11 base characters (`@`, , `$`, ...) and their two- and three-character combinations. For encoding, one of 9 fixed schemes is selected (`base64`, `rot13`, `hex`, etc.). The resulting token is then injected into the system prompt and user message for that call. Across calls the token changes, but the pool it is drawn from remains constant. Figure 3 illustrates this procedure. To ensure reproducibility, a randomization seed of 2026 (corresponding to the year of this study) was set.

### *E. Experimental Setup*

We ran the BIPIA benchmark using `deepseek-v4-flash` with and without defenses.

*1) Model:* We originally intended to run all experiments on `Gemma-4` via Radboud University's hosted LLM service. Due to the instability of the service during the evaluation period, we switched to DeepSeek[13], accessed via its API. DeepSeek was selected as a practical alternative due to its low inference cost, which made benchmark runs feasible.

*2) Test Runs:* To isolate the effect of each defense, we conducted three test runs per benchmark:

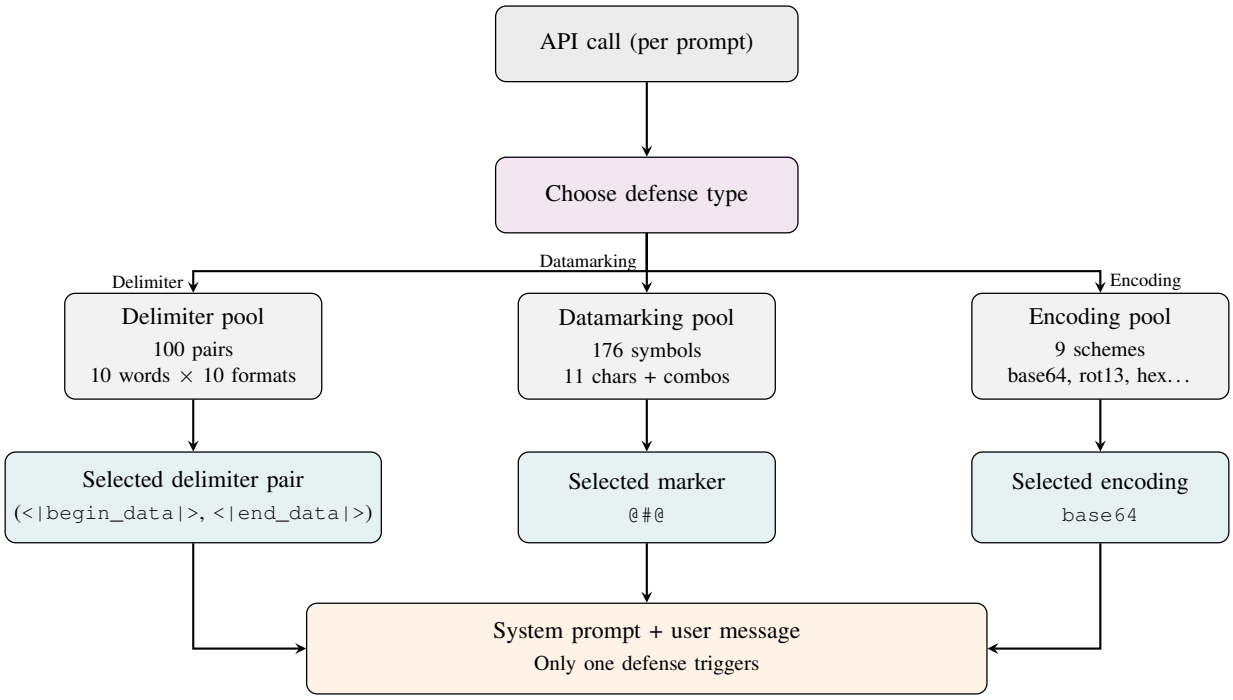



Fig. 3. Randomized Spotlighting sampling procedure. At each API call, one defense type is first sampled uniformly (delimiter, datamarking, or encoding), and then one token is drawn uniformly from that defense's pool and injected into the prompt. Only one pool is accessed per call.

1) *Baseline (plain)* in which no defense was applied. This effectively establishes the attack success rate and agent utility without any intervention.
2) *Non-randomized* The three Spotlighting techniques are applied with fixed tokens and schemes.
3) *Randomized* The same three techniques are applied with randomized tokens and schemes per call.

This structure allows us to make three comparisons:

1) *Baseline vs. non-randomized* To measure the effect of Spotlighting in general
2) *Non-randomized vs. randomized* To measure the added value of randomization
3) *Baseline vs. randomized* To assess the overall effectiveness of our approach.

## IV. Results

In this section we present the evaluation of the defenses against IPI. As we exclusively evaluated our methods on the BIPIA benchmark, the following results focus entirely on this dataset.

We defined the Attack Success Rate (ASR) as an instance when the benchmark is able to successfully manipulate the model's output. The evaluation was conducted using the `deepseek-v4-flash` model on the `email` dataset, testing a total of 1125 samples. A sample consists of one IPI attempt against the system without a defense, and the previously defined defenses.

TABLE I
Attack success rates on the BIPIA email benchmark using `deepseek-v4-flash`. Lower is better.

| Defense Strategy | Static (%) | Randomized (%) |
|---|---|---|
| Baseline | 15.38 | – |
| Delimiters | 0.53 | 0.80 |
| Encoding | 0.27 | 0.09 |
| Datamarking | 8.27 | 7.82 |

The results are summarized in Table I. and a visual comparison between static and randomized defenses is shown in Figure 4.

From these results, we observe several key findings regarding the effectiveness of the evaluated defenses:

– *Baseline vulnerability*: Without any defense mechanism the model is susceptible to IPI, exhibiting an ASR of 15.38%

– *Encoding as the strongest defense*: Encoding strategies proved to be the most robust mitigation technique. Standard encoding reduced the ASR to 0.27%, while Randomized Encoding achieved the best overall performance in our tests, reducing the

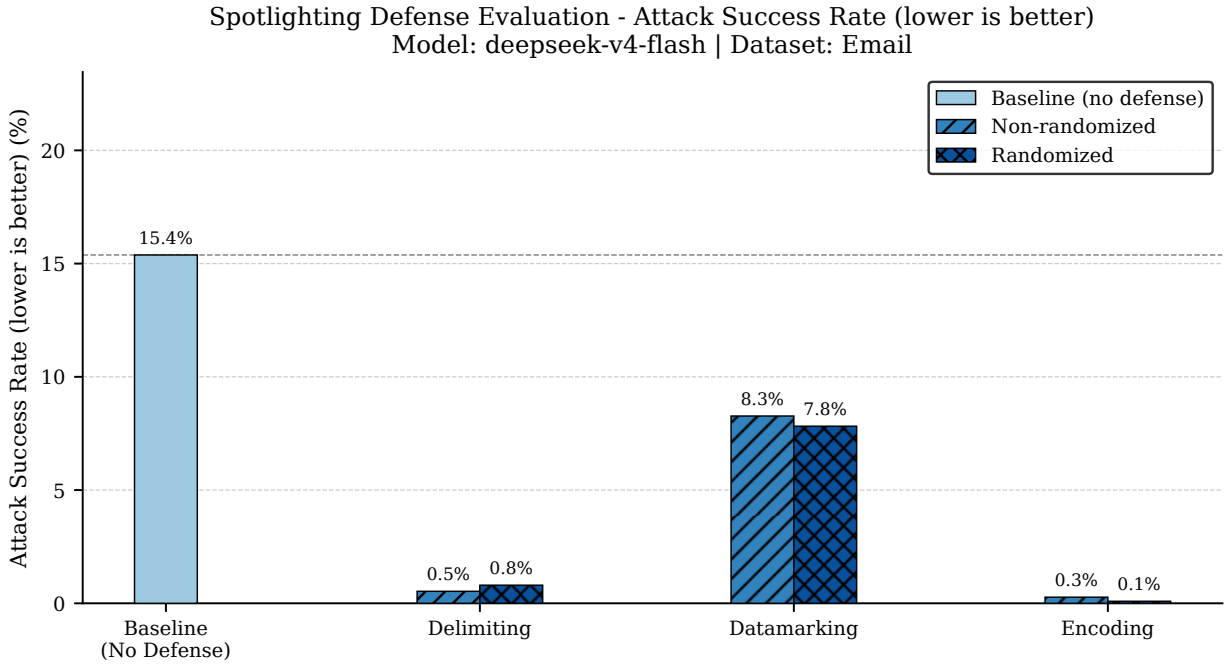



Fig. 4. Attack Success Rate (ASR) of standard and randomized Spotlighting defenses. The baseline performance (no defense) is shown on the far left, demonstrating a 15.38% ASR. Grouped bars illustrate the comparative effectiveness of non-randomized versus randomized implementations for Delimiting, Datamarking, and Encoding techniques.

ASR to a near-zero 0.09%

- *High efficacy of delimiters*: Delimiters also demonstrated excellent mitigation capabilities, dropping the ASR to 0.53%. Interestingly, randomizing the delimiters slightly increased the ASR to 0.80%, though the defense remains highly effective overall. This minimal fluctuation suggests that the model easily recognizes standard delimiters but might occasionally struggle to parse randomized boundaries.
- *Underperformance of datamarking*: Compared to the other strategies, Datamarking was the least effective, leaving an ASR of 8.27%. However, applying Randomized Datamarking yielded a marginal improvement, bringing the ASR down slightly to 7.82%.

In summary, the transition from static to randomized defenses yields varying impacts depending on the core mechanism. While *Randomized Encoding* emerges as the optimal defense strategy in this benchmark, standard static methods like static encoding and basic delimiting already provide a substantial protection against IPI attacks.

## V. Discussion

This study investigated how dynamic spotlighting influences the attack success rate (ASR) compared to static spotlighting approaches. We observed that introducing randomization to delimiter-based spotlighting slightly reduced its effectiveness (with ASR increasing from 0.53% to 0.80%), whereas encoding and data-marking approaches improved with the addition of randomness (with ASR decreasing from 0.27% to 0.09% and from 8.27% to 7.82%, respectively). Across all configurations, both the randomized and static approaches significantly outperformed the baseline of no defense.

To ground these results in a real-world context, the current-generation model evaluated (`deepseek-v4-flash`) still exhibited an attack success rate of 15.4% when no defenses were applied. Although these results should be interpreted with caution due to the small sample size, they nevertheless demonstrate that indirect prompt injection remains a highly relevant threat to modern large language models, highlighting the necessity of implementing defenses such as spotlighting.

Importantly, large language models are inherently stochastic, meaning that repeated runs of the same benchmark may yield slightly different results. To mitigate this variation as much as possible, a fixed random seed was set before running the benchmark, as described in Section III-D2. Ideally, conducting multiple benchmark runs and averaging the results would improve their

stability and reliability; however, as discussed in the Limitations (Section VI), this was not feasible due to budget constraints.

## VI. Limitations

Several limitations affected this project, many of which are interconnected.

### *A. Resource constraints.*

Large-scale benchmark evaluations required substantial computational resources and API usage, which limited the number of repeated experiments we could conduct. We initially planned on running the experiments on the shared university infrastructure hosting local LLMs with Ollama. This turned out to be unstable during our experiment. Timeout issues, interrupted requests, and routing failures prevented some planned benchmark runs from completing successfully. We had to resort to DeepSeek's paid API calls to be able to conduct the experiment within a reasonable timeframe.

### *B. Generalizability.*

The resource constraints described above directly impacted the generalizability of our findings. Within BIPIA, we only evaluated the email application scenario out of five available scenarios (code, email, QA, abstract and table), and with only 1125 (10% of the total) prompts available. While BIPIA's diversity across scenarios is designed to ensure ASR is not inflated by a narrow set of attack types, budget constraints limited us to a single scenario. Similarly, although we implemented and integrated defenses for InjecAgent and AgentDojo, we were not able to complete full-scale evaluations for these benchmarks within the available project budget and timeframe. As a result, our findings cannot be confidently extended to other scenarios or benchmarks. For completeness, the implementation details and experimental setup are documented in the appendix.

### *C. Random space limitations*

A notable limitation of our randomized Spotlighting defense is that it does not achieve true randomness. Rather than generating novel tokens at runtime, delimiters, datamarkers, and encodings are sampled from a fixed predefined pool. Consequently, an attacker with knowledge of this pool could enumerate candidate tokens and craft injections that remain effective across runs, partially undermining the intended security gain of randomization. A more robust randomness method would avoid this attack vector. For example, generating a fresh ten-digit hexadecimal string as a delimiter at each call, would substantially enlarge the effective search space. However, we deliberately did not pursue this approach in the present work, as we anticipated that introducing such tokens to the model's training distribution might degrade task performance. Whether this concern is backed by experiments, and how encoding schemes could be randomized in a principled way, remain open questions for future work.

### *D. Correctness of the output*

Furthermore, BIPIA evaluates whether an attack succeeded in hijacking the model's output, but it does not assess whether the output is sufficient for the original task. This implies that a defense can successfully block an injection, but when the response is not helpful enough, the model becomes less useful in practice. To make a more complete evaluation of the defenses, combining the ASR with a task-specific metric could be used. AgentDojo[6] addresses this by jointly evaluating the attack success and the task completion. Running this benchmark will yield more comprehensive results of the defenses and would be a good start for further research.

### *E. Future Work*

Future work could extend the evaluation to additional benchmarks and larger collections of agent-based attack scenarios. A full evaluation of InjecAgent and AgentDojo plus the four left over attack scenarios of BIPIA under both randomized and non-randomized Spotlighting defenses would provide a more complete comparison across agent environments.

Another interesting direction would be combining black-box Spotlighting defenses with white-box monitoring approaches, such as chain-of-thought analysis or anomaly detection techniques, to improve robustness against adaptive indirect prompt injection attacks.

## VII. Conclusion

This study investigated how dynamic Spotlighting influences the attack success rate (ASR) compared to static Spotlighting, in a black-box threat model where the attacker has write access to external content but no access to model internals or the system prompt.

Our results show that all Spotlighting variants, both static and randomized, reduce the ASR relative to the undefended baseline of 15.38%. Encoding proved to be the strongest defense overall, with randomized encoding achieving a near-zero ASR of 0.09%, suggesting that syntactic transformation of tool output is highly effective at preventing the model from interpreting injected content as instructions. Delimiting also performed well, though randomization marginally increased its ASR from 0.53% to 0.80%, indicating that the model may occasionally struggle to parse dynamically generated boundary tokens. Datamarking remained the weakest technique in both variants, pointing to limitations in how reliably inter-word markers signal untrusted content to the model.

With respect to our research question, the effect of randomization is mixed: it improves robustness for encoding and datamarking, but not for delimiting. This suggests that the benefit of randomization depends on the underlying mechanism, and that randomization alone is not a universal improvement. Crucially, however, all defenses, randomized or not, successfully reduced attack success against our defined attacker, who operates without knowledge of model internals. This confirms that black-box Spotlighting remains a lightweight and practically deployable defense for the threat model considered in this work.

## APPENDIX

To deploy the BIPIA benchmark to work for an API-based pipeline several changes were necessary. Originally BIPIA is heavily integrated with local GPU based serving via `vLLM`, which requires specific hardware configurations and introduces large container dependencies. We modified and patched the evaluation pipeline to run with a remote or local API, asynchronously for faster benchmarking runs.

*a) Removal of local inference dependencies:* To host the benchmark in a resource constrained environment without requiring local GPU access, we decoupled BIPIA from local inference engines:

- We modified the environment setup by dynamically removing references to `vLLM` inside the project's packaging files (`pyproject.toml` and `setup.py`).
- We stripped out all `vLLM` related imports from the execution modules, allowing the benchmark to be containerized and run in CPU only environments without local GPU compilers or massive CUDA libraries.

*b) Asynchronous API integration:* We transitioned the evaluation runner from synchronous local generation loops to an asynchronous loop using `asyncio` and `aiohttp`:

- The model calls are routed to an OpenAI compatible endpoint. This model supports querying both remote APIs and local model runners (Ollama, LiteLLM, etc).
- To avoid rate limiting or overloading target endpoints, requests are managed using a semaphore for configurable concurrency control.
- A robust retry handler with exponential backoff and random jitter was added to handle network timeouts or error codes suggesting API instability.

*c) Runtime library patching:* The original BIPIA dataset loading utilities rely on, among others, PyTorch. Because these components introduce environment conflicts, these are dynamically patched out during runtime:

- During initialization our script overrides the contents of `/app/BIPIA/bipia/data/utils.py` with simplified Python string insertions.
- It replaces the heavy dataset collators with lightweight mock classes. This allows for standard, no-dependency, dictionary-based dataset parsing and avoids data tokenization errors without altering the evaluation data.

*d) Robustness patch for defense randomization:* When evaluating the randomized defenses described in Section III-D, the `DefenseRandomizer` can select text transformation encodings (such as ROT47 or Morse code) that may not be natively supported by the string conversion logic in the baseline `Defenses.encoding` class. To prevent execution crashes due to unhandled `ValueError` exceptions, we dynamically patch the encoding system:

- We intercept calls to the static encoding method at runtime.
- If the randomized defense selects an encoding scheme that cannot be evaluated, the patch catches the resulting error and falls back to the standard Base64 encoding scheme.

*e) Incremental progress tracking and signal interception:* To safeguard against lost evaluation data during long-running benchmark jobs, we implemented defensive logging and processing monitoring:

- Evaluation results are streamed incrementally to a local JSONL file as they complete, rather than being accumulated solely in system memory.
- we registered signal interception hooks for both `SIGTERM` and user interrupts `KeyboardInterrupt`. If a job is stopped prematurely, the runner catches the signal, compiles the current performance metrics (including completed block rates and query statistics), and outputs a finalized `comparison_summary.json` file before terminating.

## REFERENCES

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*,

ser. AISec '23, New York, NY, USA: Association for Computing Machinery, Nov. 26, 2023, pp. 79–90, ISBN: 979-8-4007-0260-0. DOI: 10.1145/3605764.3623985. Accessed: Mar. 29, 2026. [Online]. Available: https://dl.acm.org/doi/10.1145/3605764.3623985.

[2] L. Shi, Y. Kang, J. Hu, X. Li, and M. Yang, "Meticulous Thought Defender: Fine-Grained Chain-of-Thought (CoT) for Detecting Prompt Injection Attacks of Large Language Models," *IEEE Access*, vol. 13, pp. 113 194–113 207, 2025, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2025.3583759. Accessed: May 22, 2026.

[3] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kıcıman, "Defending Against Indirect Prompt Injection Attacks With Spotlighting,"

[4] J. Yi et al., "Benchmarking and Defending against Indirect Prompt Injection Attacks on Large Language Models," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, ser. KDD '25, New York, NY, USA: Association for Computing Machinery, Jul. 2025, pp. 1809–1820, ISBN: 979-8-4007-1245-6. DOI: 10.1145/3690624.3709179. Accessed: Mar. 29, 2026.

[5] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, L.-W. Ku, A. Martins, and V. Srikumar, Eds., Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 10 471–10 506. DOI: 10.18653/v1/2024.findings-acl.624. Accessed: Apr. 7, 2026.

[6] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. [Online]. Available: https://openreview.net/forum?id=m1YYAQjO3w.

[7] *LLMail-Inject: Adaptive Prompt Injection Challenge*, https://microsoft.github.io/llmail-inject/, 2024. Accessed: May 22, 2026.

[8] F. Perez and I. Ribeiro, *Ignore previous prompt: Attack techniques for language models*, 2022. arXiv: 2211.09527 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2211.09527.

[9] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," *arXiv preprint arXiv:2302.12173*, 2023. DOI: 10.48550/arXiv.2302.12173. [Online]. Available: https://arxiv.org/abs/2302.12173.

[10] S. Abdelnabi, A. Fay, G. Cherubin, A. Salem, M. Fritz, and A. Paverd, "Are you still on track!? Catching LLM task drift with activations," *arXiv preprint arXiv:2406.00799*, 2024.

[11] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," *arXiv preprint arXiv:2404.13208*, 2024.

[12] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, *Universal and transferable adversarial attacks on aligned language models*, 2023. arXiv: 2307.15043 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2307.15043.

[13] DeepSeek-AI, *Deepseek-v4: Towards highly efficient million-token context intelligence*, https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash, 2026.